

# Productivity Dashboard Documentation

## Project Overview

---

This project is a single-page productivity web app built using core web technologies (structure, styling, and interactivity). It brings together several small daily-use tools into one dashboard, so a user can manage tasks, plan their day, stay motivated, track focus time, and check the weather - all from one screen.

**Main Objective:** Practice DOM manipulation, event handling, and browser storage by building multiple independent mini-features that work together inside one dashboard layout.

### Features Included

- Dashboard Navigation
  - Todo List
  - Daily Planner
  - Motivation Quote
  - Pomodoro Timer
  - Weather Widget
  - Date & Time Display
  - Dynamic Background
  - Theme Switch (Light/Dark)
  - Daily Goals
- 

## Project Workflow

---

The app follows a simple, repeating flow:

1. Dashboard loads - the user sees cards/tiles for each feature (Todo, Planner, Motivation, Pomodoro, Goals) along with always-visible widgets (Date & Time, Weather).
2. User selects a feature - clicking a card opens that feature.
3. Feature opens in full view - the selected feature takes over the main view so the user can focus on it.
4. User interacts with it - adding tasks, starting a timer, marking goals complete, etc.
5. Data is stored or updated - changes are saved (e.g., in Local Storage) so they aren't lost on refresh.
6. User returns to dashboard - a back/close action brings the user back to the main screen to pick another feature.

This pattern (dashboard → open feature → interact → save → return) repeats for every feature, so once you build it once, the rest follow the same logic.

---

## Feature Documentation

---

### Dashboard Navigation

#### Objective

Acts as the home screen and traffic controller of the app. It lets the user see all available tools and switch between them without reloading the page.

## What Students Need to Build

- A grid/list of clickable cards, one per feature.
- A "full screen" container that can display whichever feature is currently open.
- A way to go back to the dashboard from any open feature.

## Implementation Steps

1. Design the dashboard layout with one card per feature (icon/image + title).
2. Design a separate section/container for the "opened feature" view.
3. Keep the feature view hidden by default.
4. Add a click listener on each card.
5. On click, hide the dashboard and show the matching feature section.
6. Add a back/close button inside each feature view.
7. On back/close click, hide the feature view and show the dashboard again.
8. Make sure only one feature is visible at a time.

## Concepts Used

- DOM Manipulation
- Event Listeners
- Dynamic Rendering (show/hide sections)

## Challenge

- Keep track of which section is currently active so two features don't overlap.
- Make navigation reusable - avoid writing separate show/hide logic for every single card.
- Handle the case where a user clicks the same card twice quickly.

---

## Todo List

### Objective

Lets the user create a personal task list, mark tasks as important, and track completion - the core "productivity" feature of the app.

## What Students Need to Build

- An input field and "Add Task" button.
- A list area where tasks appear.
- A way to mark a task as important.
- A way to mark a task as complete and delete a task.

## Implementation Steps

9. Design the input field, add button, and task list container.
10. Select the input and button elements in JavaScript.
11. Add an event listener to the "Add Task" button (and Enter key, optionally).
12. On submit, read the input value and validate it's not empty.
13. Create a new task item and add it to the list in the DOM.
14. Add buttons/icons on each task for "mark important," "complete," and "delete."
15. Attach event listeners to these controls (event delegation works well here).

16. Save the current task list to Local Storage whenever it changes.
17. On page load, read from Local Storage and rebuild the list.
18. Clear the input field after adding a task.

### Concepts Used

- DOM Manipulation
- Event Listeners
- Local Storage
- Dynamic Rendering

### Challenge

- Keep task data (text, completed, important) and the way it's displayed in sync.
  - Use event delegation instead of adding a listener to every single task.
  - Don't lose tasks on page refresh - test Local Storage saving/loading carefully.
- 

## Daily Planner

### Objective

Helps the user organize their day by assigning tasks or notes to specific time slots.

### What Students Need to Build

- A layout showing time slots (e.g., hourly blocks) for the day.
- An input method to add a plan/note to a slot.
- Persistent storage so the plan isn't lost.

### Implementation Steps

19. Design a list of time slots covering the hours of a day.
20. Decide the data structure: one entry per time slot (time + text).
21. Select each time slot's input/edit area in JavaScript.
22. Add an event listener to save when the user types or submits.
23. Store the updated plan in Local Storage.
24. On page load, read stored data and fill in each time slot.
25. Allow clearing or editing an existing entry.
26. Handle empty slots gracefully (don't show broken/undefined text).

### Concepts Used

- DOM Manipulation
- Event Listeners
- Local Storage
- Date Object (optional, for auto-highlighting the current hour)
- Dynamic Rendering

### Challenge

- Keep the time slots in the correct order.
- Decide how/when to save - on every keystroke or only when the user finishes typing.

- Optionally highlight the current time slot using the Date Object.
- 

## Motivation Quote

### Objective

Displays an inspirational quote to the user, fetched live from an API, to keep them motivated while working.

### What Students Need to Build

- A display area for the quote text (and optionally the author).
- A button to fetch and show a new quote.

### Implementation Steps

27. Design the quote card with space for text and a "new quote" button.
28. Find and sign up for a free quote API (get an API key if required).
29. Select the quote display elements in JavaScript.
30. Add an event listener to the "new quote" button (and optionally trigger it once on page load).
31. On click, use the Fetch API to request a new quote from the API.
32. Wait for the response and parse the JSON data.
33. Update the DOM with the new quote text (and author, if provided).
34. Show a loading state while the request is in progress.
35. Handle errors (e.g., API down, network issue) with a fallback message so the UI doesn't break.

### Concepts Used

- DOM Manipulation
- Event Listeners
- Fetch API
- Dynamic Rendering

### Challenge

- Handle the delay between clicking the button and the quote arriving - don't leave the card blank.
  - Handle network/API failures gracefully instead of breaking the page.
  - Keep the quote card readable even if the returned quote text is very long.
- 

## Pomodoro Timer

### Objective

A focus timer based on the Pomodoro technique - the user works for a set duration (e.g., 25 minutes), then takes a break, helping them manage time in focused sessions.

### What Students Need to Build

- A countdown display (minutes:seconds).
- Start, Pause, and Reset controls.
- A label showing the current session type (e.g., "Work Session").

### Implementation Steps

36. Design the timer display and the Start/Pause/Reset buttons.
37. Decide the default work duration (e.g., 25 minutes) in seconds.
38. Select the timer display and buttons in JavaScript.
39. On "Start," begin a repeating timer using setInterval.
40. Every tick, decrease the remaining time by one second and update the display.
41. On "Pause," stop the interval without resetting the time.
42. On "Reset," stop the interval and set the time back to the default.
43. When the countdown reaches zero, stop the timer and notify the user (sound/alert/message).
44. Format the remaining seconds into MM:SS before displaying.
45. Prevent multiple intervals from running at the same time.

## Concepts Used

- Timers (setInterval / clearInterval)
- DOM Manipulation
- Event Listeners
- Dynamic Rendering

## Challenge

- Clear the previous interval before starting a new one to avoid the timer running too fast.
  - Correctly convert total seconds into minutes and seconds for display.
  - Decide what happens after a work session ends (auto-start break, or wait for user).
- 

## Weather Widget

### Objective

Shows current weather conditions (temperature, condition, humidity, wind, etc.) for the user's location, adding real-world, real-time data to the dashboard.

### What Students Need to Build

- A display area for location name, temperature, condition, and details (precipitation, humidity, wind).
- Logic to fetch this data from a weather API.

### Implementation Steps

46. Design the weather card layout with placeholders for each data point.
47. Sign up for a free weather API and get an API key.
48. Get the user's location (via the Geolocation API or a fixed/default city).
49. Use the Fetch API to request weather data using the location.
50. Wait for the response and parse the JSON data.
51. Select the relevant DOM elements for each weather value.
52. Update the DOM with the fetched values (temperature, condition, etc.).
53. Handle the loading state while the request is in progress.
54. Handle errors (e.g., location denied, API failure) with a fallback message.

## Concepts Used

- Fetch API

- DOM Manipulation
- Dynamic Rendering

## Challenge

- Handle the delay between requesting data and receiving it (don't leave the UI blank/broken).
  - Handle cases where the user denies location access.
  - Match the icon/condition text to the correct weather description from the API response.
- 

## Date & Time

### Objective

Displays the current date and live-updating time on the dashboard, giving the user constant context while they work.

### What Students Need to Build

- A display for the current date (day, date, month, year).
- A display for the current time that updates continuously.

### Implementation Steps

55. Design a small area on the dashboard for date and time text.
56. Select the display elements in JavaScript.
57. Create a function that gets the current date/time using the Date Object.
58. Format the date and time into a readable string.
59. Update the DOM with this formatted string.
60. Use setInterval to re-run this function every second (or minute).
61. Call the function once immediately on page load, so it doesn't wait a full second to appear.

### Concepts Used

- Date Object
- Timers (setInterval)
- DOM Manipulation

## Challenge

- Format time correctly (12-hour vs 24-hour, leading zeros for single-digit minutes).
  - Avoid creating multiple overlapping intervals if this code runs more than once.
  - Keep the update frequency reasonable (every second is usually enough).
- 

## Dynamic Background

### Objective

Changes the dashboard's background (e.g., image, gradient, or theme) based on context - such as time of day - to make the app feel more alive and personalized.

### What Students Need to Build

- A set of background options tied to a condition (e.g., morning/afternoon/evening/night).
- Logic to pick and apply the right background automatically.

## Implementation Steps

62. Collect or design a background image/gradient for each time-of-day category.
63. Decide the time ranges for each category (e.g., 5–11 AM = morning).
64. Get the current hour using the Date Object.
65. Write a condition that matches the current hour to a category.
66. Select the element whose background should change.
67. Apply the matching background using DOM/CSS updates.
68. Re-check and update this periodically in case the app stays open across a time boundary.

## Concepts Used

- Date Object
- DOM Manipulation
- CSS Variables
- Timers (optional, for periodic re-check)

## Challenge

- Cover all 24 hours without gaps in your time-range conditions.
  - Make sure text remains readable on every possible background.
  - Avoid abrupt visual "flashing" when the background updates.
- 

## Theme Switch

### Objective

Lets the user toggle between light and dark modes based on personal preference, and remembers their choice for future visits.

### What Students Need to Build

- A toggle button/switch.
- Two visual themes (light and dark) defined in CSS.
- Persistence of the selected theme.

## Implementation Steps

69. Define light and dark color values using CSS Variables.
70. Design a toggle control (button or switch) on the dashboard.
71. Select the toggle element and the root element that holds the theme in JavaScript.
72. Add an event listener to the toggle.
73. On toggle, switch the active theme by changing a class or CSS variable set.
74. Save the selected theme to Local Storage.
75. On page load, check Local Storage and apply the saved theme immediately.

## Concepts Used

- CSS Variables
- DOM Manipulation
- Event Listeners
- Local Storage

## Challenge

- Make sure every part of the UI (not just the background) responds to the theme change.
  - Apply the saved theme before the page visibly renders, to avoid a "flash" of the wrong theme.
  - Keep the toggle's visual state in sync with the actual active theme.
- 

## Daily Goals

### Objective

Lets the user set short-term goals for the day and track which ones are completed, giving a clear sense of daily progress.

### What Students Need to Build

- An input to add a new goal.
- A list of goals with a way to mark each as done.
- A simple progress indicator (e.g., "2 of 5 completed").

### Implementation Steps

76. Design the input, "add goal" button, and goal list container.
77. Select these elements in JavaScript.
78. Add an event listener to add a new goal from the input.
79. Render each goal in the list with a checkbox or complete button.
80. Add an event listener (using delegation) to toggle a goal's completed state.
81. Recalculate and display the progress count whenever goals change.
82. Save the goals list to Local Storage on every change.
83. On page load, load saved goals and rebuild the list and progress count.

### Concepts Used

- DOM Manipulation
- Event Listeners
- Local Storage
- Dynamic Rendering

## Challenge

- Keep the progress count accurate every time a goal is added, removed, or toggled.
  - Reuse the same list/storage pattern you built for the Todo List where possible.
  - Decide how completed goals should look differently from pending ones.
- 

## Project Structure

---

- **index.html** - Contains the structure of the dashboard and all feature sections (Todo, Planner, Motivation, Pomodoro, Weather, Goals, etc.). Acts as the skeleton the other files style and animate.
- **style.css** - Handles the visual appearance: layout, colors, light/dark theme variables, and responsive design.
- **script.js** - Contains all the logic: navigation between features, event listeners, Local Storage handling, API calls, and timers.

- **assets folder (images/videos/fonts)** - Stores images, icons, preview videos, and custom fonts used across the dashboard and feature cards.
- 

## Browser Features Used

---

- **Local Storage** - Used to save Todo tasks, Daily Planner entries, Daily Goals, and the selected theme, so data persists after a page refresh.
  - **Fetch API** - Used to get live data from external sources: the Weather Widget (weather API) and the Motivation Quote feature (quote API).
  - **setInterval** - Powers the Pomodoro Timer countdown and the live-updating Date & Time display.
  - **Date Object** - Used to get the current date/time for the Date & Time display and to decide which background to show in the Dynamic Background feature.
- 

## Suggested Build Order

---

1. Create UI (basic HTML structure and styling for the dashboard and all feature sections)
  2. Navigation (show/hide logic between dashboard and features)
  3. Todo List
  4. Daily Planner
  5. Daily Goals
  6. Pomodoro Timer
  7. Motivation Quotes
  8. Weather
  9. Date & Time
  10. Theme Switch
  11. Final Polish (responsiveness, animations, edge cases, cleanup)
- 

## Learning Outcomes

---

By completing this project, students will be able to:

- Build multi-section, single-page applications using DOM manipulation.
- Manage application state and persist it using Local Storage.
- Fetch and display real-time data from external APIs.
- Work with timers to build countdowns and live clocks.
- Use the Date Object to build time-aware features.
- Implement a light/dark theme system using CSS Variables.
- Structure a growing JavaScript project into organized, reusable logic.